Two kinds of rules, in plain English

Rule languages · Logical English · Logic Production Systems

Robert Kowalski · Imperial College London

JURIX 2026 tutorial · Toulouse · 8 December 2026



A notice on the London Underground

Emergencies

Press the alarm signal button to alert the driver.

The driver will stop if any part of the train is in a station.

If not, the train will continue to the next station, where help can more easily be given.

There is a 50 pound penalty for improper use.

Millions of passengers have read it. It is a program, written to be executed by people.



1 · Rule languages

Two kinds of rules



The notice has two kinds of sentence

WHAT IS THE CASE

> The driver will stop if any part of the train is in a station.

It describes the world.

It is true or it is false.

It is a **belief**.

WHAT TO DO

> Press the alarm signal button to alert the driver.

It does not describe the world.

It asks for the world to be changed.

It is a **goal**.

Belief rules

conclusion **if** *conditions*

```
the train stops in a station
    if the driver is alerted
    and a part of the train is in the station.
```

- A belief rule defines its conclusion in terms of its conditions
- A set of belief rules is a **logic program**
- The same rule is used **backwards**, to answer a question, and **forwards**, to derive a consequence



Goal rules

if *conditions* **then** *conclusion*

```
if there is an emergency
then the driver is alerted.
```

```
it must not be true that
    the driver stops the train
    and it is not the case that the train is in a station.
```

- A goal rule does not define anything. It must be **made true**, by acting
- The first kind is a **reactive rule**. The second kind is a **constraint**
- An obligation is a goal. A prohibition is a constraint



Goals say what. Beliefs say how.

THE GOAL

```
if there is an emergency
then the driver is alerted.
```

A BELIEF

```
the driver is alerted
    if a passenger presses
       the alarm signal button.
```

THE REASONING

1. There is an emergency. *An observation.*
2. So the driver must be alerted. *Forwards, with the goal.*
3. The driver is alerted if I press the button. *Backwards, with the belief.*
4. I press the button. *An action.*

"Press the alarm signal button to alert the driver" is the belief, written backwards.



Rule languages take sides

Mostly belief rules	Mostly goal rules
Prolog, Datalog, answer set programming, s(CASP)	Production rules: Drools
Catala, OpenFisca, RuleSpec	Event-condition-action rules
Decision tables	Obligations and prohibitions: LegalRuleML, L4
Blawx, Epilog, Oracle Intelligent Advisor	Smart contracts: Solidity, Daml

A language with one kind of rule must imitate the other kind.

Logical English writes belief rules. **Logic Production Systems** adds goal rules, and time.



2 · Controlled natural language and Logical English

English that a computer can read as logic

English is expressive. It is also ambiguous.

> The driver will stop if any part of the train is in a station.
>
> If not, the train will continue to the next station.
>
> There is a 50 pound penalty for improper use.

- Stop **what**? And **where**?
- If **not** what?
- Improper use **of what**?

A CONTROLLED NATURAL LANGUAGE

A part of English with one reading only.

LOGICAL ENGLISH

- *conclusion* **if** *conditions*
- No pronouns: **a** train, then **the** train
- One word, one meaning
- Time, but no tense

It is another way to write a logic program.



Templates

```
the templates are:
    *a person* presses the alarm signal button of *a train*.
    the driver of *a train* is alerted.
    *a part* of *a train* is in *a station*.
    *a train* stops in *a station*.
    *a train* continues to the next station.
```

- A **template** is the pattern of a sentence
- The words between asterisks are **places**. Each place stands for one thing
- In logic, a template is a predicate, and a place is an argument



The notice, in Logical English

```
the driver of a train is alerted
    if a person presses the alarm signal button of the train.

a train stops in a station
    if the driver of the train is alerted
    and a part of the train is in the station.

a train continues to the next station
    if the driver of the train is alerted
    and it is not the case that
        a part of the train is in a station.
```

a train introduces a variable. **the** train refers back to it. Nothing else is needed.



A scenario is a case. A query is a question.

```
scenario between stations is:
    alice presses the alarm signal button of the 8:15.
    stops expects answers [].
    continues expects answers ["the 8:15 continues to the next station"].

query stops is:
    which train stops in which station.

query continues is:
    which train continues to the next station.
```

- A **scenario** gives the facts of one case
- A scenario can say which answers it **expects**. Then the scenario is also a test



Every answer has an explanation

WHY?

```
the 8:15 continues to the next station
  the driver of the 8:15 is alerted
    alice presses the alarm signal
        button of the 8:15
  it is not the case that
    a part of the 8:15 is in a station
```

WHY NOT?

```
not: a train stops in a station
  the driver of the 8:15 is alerted
    alice presses the alarm signal
        button of the 8:15
  not: a part of the 8:15
        is in a station
```

The explanation is the proof. It uses the sentences of the program, so the reader of the rules can read it.



The fourth sentence: a citation, and a conditional answer

```
the templates are:
    *a person* is liable to a penalty of *an amount*.
    *a person* uses the alarm signal button improperly; unknown.

rule sentence_4 with provenance the Emergency Notice,
        confer "There is a 50 pound penalty for improper use":
a person is liable to a penalty of 50
    if the person presses the alarm signal button of a train
    and the person uses the alarm signal button improperly.
```

Answer: alice is liable to a penalty of 50, **assuming** alice uses the alarm signal button improperly.

The notice does not say what is improper. The program does not pretend to know.



3 · Logic Production Systems

Rules in a world that changes



The notice is about change

```
the events are:
    a fire starts.
    the train leaves the station.
    the train arrives at the next station.

the actions are:
    *a person* presses the alarm signal button.
    the driver stops the train.
    *a person* pays a penalty of *an amount*.

the fluents are:
    there is an emergency.
    *a person* is on the train.
    the driver is alerted.
    the train is in a station.
    the train is stopped.
```

- A **fluent** is a fact that changes over time
- An **event** happens, and changes the fluents
- An **action** is an event that an agent chooses

The same language. One new first line:

```
the target language is: lps.
```



Causal laws: what an event does

```
when a fire starts
then there is an emergency.

when a person presses the alarm signal button
then the driver is alerted.

when the train leaves the station
then it is not the case that the train is in a station.

when the train arrives at the next station
then the train is in a station.
```

- A **causal law** says which fluents an event starts, and which fluents it ends
- Causal laws are beliefs. A fluent that no event changes stays as it is



Goals: two reactive rules and a constraint

```
if there is an emergency
    and a person is on the train
    and it is not the case that the driver is alerted
then the person presses the alarm signal button.

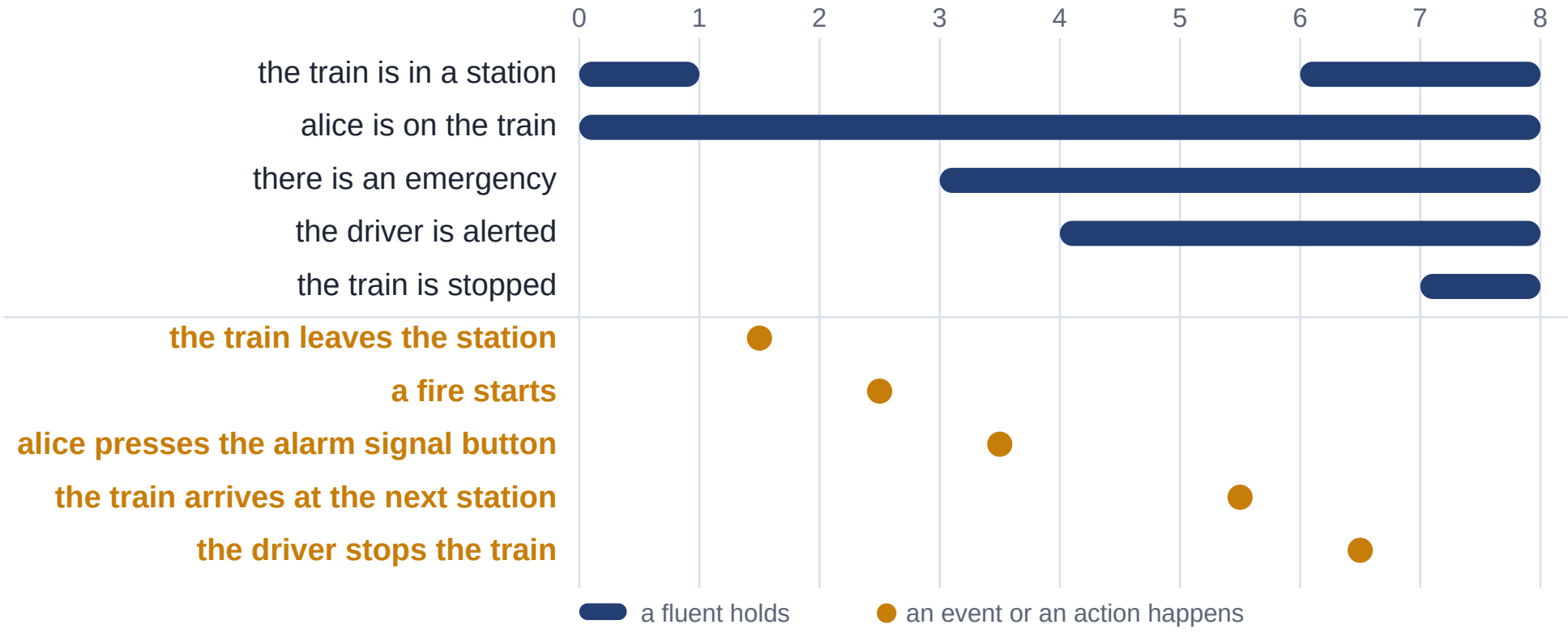
if the driver is alerted at a first time
    and it is not the case that the train is stopped at the first time
then the driver stops the train from a second time to a third time.

it must not be true that
    the driver stops the train
    and it is not the case that the train is in a station.
```

The driver must stop the train **at some later time**. The constraint says **when the driver may**.



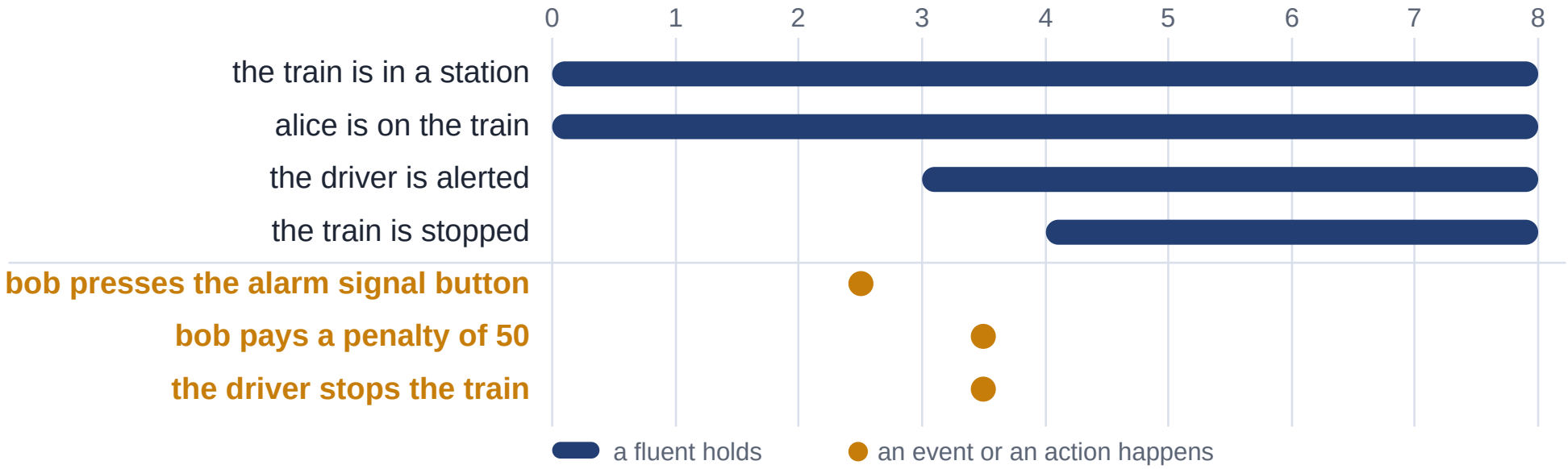
A fire between two stations



The events come from outside. The rules choose the actions. The timeline is the model that makes the goals true.

The penalty is a goal too

```
if it is not the case that there is an emergency
    and a person presses the alarm signal button
then the person pays a penalty of 50.
```



An obligation is a reactive rule: when its conditions happen, an action must follow.

What to take away

- **Belief rules** say what is the case: *conclusion* if *conditions*
- **Goal rules** say what must be made true: if *conditions* then *conclusion*
- **Logical English** writes rules in English that has one reading only
- **Logic Production Systems** makes the goals true in a world that changes

Next: sixteen other rule languages, read in the same English.

